HDOC Day-67 Activity

GCD of Two Positive Integers Using Recursive Euclid's Algorithm

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-67

Aim

To write a recursive C program that finds the **Greatest Common Divisor (GCD)** of two positive integers using **Euclid's algorithm**.

What is GCD?

The GCD of two positive integers is the largest positive integer that divides both numbers exactly, without leaving a remainder. For example, the GCD of 48 and 18 is 6.

Euclid's Algorithm

Euclid's algorithm uses the fact that the GCD does not change when the larger problem **GCD(a, b)** is replaced by the smaller problem **GCD(b, a % b)**. The process continues until the second number becomes 0.

Problem Requirements

- Read two positive integers from the user.
- Use a recursive function to calculate their GCD.
- Apply Euclid's algorithm.
- Stop recursion when the second number becomes 0.
- Display the GCD as an integer.

Algorithm

1. Read two positive integers a and b.
2. Call the recursive function **gcd(a, b)**.
3. Base case: if b == 0, return a.
4. Otherwise recursively call **gcd(b, a % b)**.
5. Each call reduces the problem because the remainder is smaller than the divisor.
6. When the base case is reached, return the GCD and display it.

C Program

```c
#include <stdio.h>

int gcd(int a, int b)
{
    if (b == 0)
    {
        return a;
    }

    return gcd(b, a % b);
}

int main()
{
    int a, b;

    printf("Enter two positive integers: ");
    scanf("%d %d", &a, &b);

    if (a <= 0 || b <= 0)
    {
        printf("Please enter positive integers.");
        return 0;
    }

    printf("GCD: %d", gcd(a, b));

    return 0;
}
```

Base Case

The statement **if (b == 0) return a;** is the base case. When the remainder becomes zero, the current value of a is the greatest common divisor. Without this base case, the recursive calls would not terminate.

Dry Run - GCD(48, 18)

Recursive Call	Remainder	Next Call
gcd(48, 18)	48 % 18 = 12	gcd(18, 12)
gcd(18, 12)	18 % 12 = 6	gcd(12, 6)
gcd(12, 6)	12 % 6 = 0	gcd(6, 0)
gcd(6, 0)	Base case	Return 6

Therefore, **GCD(48, 18) = 6**.

Sample Test Cases

a	b	GCD
48	18	6
36	24	12
25	15	5

17	13	1
100	25	25

How Recursion Reduces the Problem

Every recursive call replaces (a, b) with (b, a % b). Since the remainder is always smaller than b, the numbers quickly decrease until the remainder becomes zero. This makes Euclid's algorithm much faster than checking every possible divisor.

Complexity

Time Complexity: O(log(min(a, b))) in the standard analysis of Euclid's algorithm.

Space Complexity: O(log(min(a, b))) due to the recursive call stack.

Result

The program successfully finds the GCD of two positive integers using recursive Euclid's algorithm. The recursion terminates when the second number becomes zero, and the remaining first number is returned as the GCD.

Execution Example

```
Enter two positive integers: 48 18
GCD: 6
```